

Lab1 实验报告

张雪阳 25301030086

```
root@ics:~/Lab1# ./btest
Score   Rating  Errors  Function
1       1       0       signMask
2       2       0       bitXor
3       3       0       negativePart
4       4       0       copyByteWithin
4       4       0       logicalShift
4       4       0       swapNibblePairs
4       4       0       secondLowestZeroBit
5       5       0       oddParity
5       5       0       rotateRightBits
5       5       0       roundEvenPow2
5       5       0       midpointTowardFirst
7       7       0       isBetweenEitherOrder
7       7       0       mul5Sat
7       7       0       classifyAdd3
7       7       0       floatScaleThreeHalves
10      10      0       floatRoundEven
10      10      0       float_i2f
10      10      0       bitCount
10      10      0       bitReverse
Total points: 110/110
```

```
root@ics:~/Lab1# ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 5/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 5/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 19/24 operators
midpointTowardFirst: 23/32 operators
isBetweenEitherOrder: 47/48 operators
mul5Sat: 14/30 operators
classifyAdd3: 29/52 operators
floatScaleThreeHalves: 34/60 operators
floatRoundEven: 16/65 operators
float_i2f: 25/40 operators
bitCount: 28/40 operators
bitReverse: 29/34 operators
All 19 functions passed operator checks.
```

P1 思路

32 位 int 中，数字 1 二进制只有最低位是 1。使用左移运算符<<，把 1 向左移动 31 位。移位之后原本最低位的 1 被移动到第 31 位（最高符号比特位），其余比特全部补 0，直接得到只有符号位为 1 的掩码。

P2 思路

按位异或是把数字转成二进制，一位对一位的单独比较，若相同取 0，不同取 1。

只使用~和&的话，

- ①用 $x \& y$ 找出两边同时为 1 的 bit，再在外面套上~，变成 $\sim(x \& y)$ 将这些位置翻转为 0
- ②用 $\sim x \& \sim y$ 找出两边同时为 0 的 bit； $\sim(\sim x \& \sim y)$ 将这些位置翻转为 1
- ③用&连接①和②的结果，这样一边 0 一边 1 “的 bit 会保留为 1

P3 思路

使用算术右移：int 有 32 位，负数右移 31 位，会得到全 1 掩码 0xFFFFFFFF；正数右移 31 位得到全 0 掩码 0x00000000。

①令 $\text{int mask} = x \gg 31$;

②如果 $x \geq 0$ ，那么 $\text{mask} = 0$ ，后续 $\text{mask} \& \text{任意数}$ 结果直接为 0，直接达成返回 0 的要求。

如果 $x < 0$ ， $\text{mask} = 0xFFFFFFFF$ ， $\text{mask} \& \text{任意数} = \text{原本任意数}$ ，保留数值，需要进一步得到 $-x$ 。

③普通取反公式 $-x = \sim x + 1$ ，但是直接写 $\sim x + 1$ ，当输入为最小负数 0x80000000，-INT_MIN 超出 32 位 int 能表达的范围，发生有符号整数溢出，属于 C 语言未定义行为，测试时发生了报错经过改进，把结果拆成两部分相加： $(\text{mask} \& (\sim x)) + (\text{mask} \& 1)$

当 x 为负数时 mask 全 1： $\text{mask} \& \sim x = \sim x$ ， $\text{mask} \& 1 = 1$ ；相加等价于 $\sim x + 1$ ，得到 $-x$ 。

当 x 非负时 mask 全 0：两项全部为 0，相加结果等于 0。

P4 思路

题目想要把源字节的 8 比特写入 dst 字节位置，首先要拿到源 src 字节的 8 比特数据，所以把原始数字 x 右移，把 src 对应的字节挪到最低字节位置，使用 0xFF，保留最低 8 位，提取出源字节的内容。

然后需要把这 8 比特移动到目标 dst 字节对应的比特位置，对提取得到的字节执行左移位，将它搬运到目标字节的位置。

因为不能直接把数据写入目标位置（容易造成新旧数据混杂），所以需要先把原始数字中 dst 位置的 8 比特全部清零，其他字节的比特保留。构造一个掩码，使之在目标 dst 字节的 8 比特处全部置 0，其余全部比特置 1。将原数字和掩码做按位与运算，清空目标字节。

目标字节已经清空，将搬运到目标位置的源字节数据，和清理完成的数字做按位或运算，把复制得到的字节写入目标位置，得到最终结果。

P5 思路

在算术右移中 $x \gg n$ 负数左边补 1；正数左边补 0。

而逻辑右移无论正负，左边空位永远补 0。

先做算术右移： $x \gg n$ 。负数高位会补一堆 1。

然后造一个掩码：把最左边的 n 个 bit 清零，剩下 bit 全部保留

最后 $(x \gg n) \& \text{mask}$ ，把高位多余的 1 清成 0，就得到逻辑右移结果。

P6 思路

题干要求输入 32 位整数，对每一字节，交换字节内部高 4 比特与低 4 比特，跨字节比特互不干扰。

所以先构造掩码 0x0F0F0F0F， $(x \gg 4) \& 0x0F0F0F0F$ ：把每个字节的高 4 位挪到低 4 位位置，用掩码滤掉多余 bit。 $(x \& 0x0F0F0F0F) \ll 4$ ：把每个字节原来的低 4 位挪到高 4 位位置。

|按位或，把两部分拼起来，每个字节完成高低 4 位互换。

使用./check_ops.py bits.c 检查时出现了报错，0x0F0F0F0F 超出允许范围，是非法常数。所以我先用单字节合法常量 0x0F，通过左移拼出完整 0x0F0F0F0F

```
Operator check failed:
swapNibblePairs:
dlc:/tmp/tmpnn6ai9hd.c:2:swapNibblePairs: Illegal constant (0x0F0F0F0F) (only 0x0 - 0xff allowed)
dlc:/tmp/tmpnn6ai9hd.c:3:swapNibblePairs: Illegal constant (0x0F0F0F0F) (only 0x0 - 0xff allowed)
```

(改前)

```
root@ics:~/Lab1# ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 5/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 5/18 operators
secondLowestZeroBit: 0/8 operators
oddParity: 0/56 operators
rotateRightBits: 0/16 operators
roundEvenPow2: 0/24 operators
midpointTowardFirst: 0/32 operators
isBetweenEitherOrder: 0/48 operators
mul5Sat: 0/30 operators
classifyAdd3: 0/52 operators
floatScaleThreeHalves: 0/60 operators
floatRoundEven: 0/65 operators
float_i2f: 0/40 operators
bitCount: 0/40 operators
bitReverse: 0/34 operators
All 19 functions passed operator checks.
```

(改后)

P7 思路

题干要求输入 32 位整数，从最低位 bit0 向高位查找，找出第二个值为 0 的比特位，返回该比特对应的掩码，若不存在第二个 0 则返回 0。

所以先执行 $z = \sim x$ 做按位取反，把寻找原数中比特 0 的问题转化为寻找取反结果 z 中的比特 1。表达式 $a \& (\sim a + 1)$ 可以提取一个数最右侧置 1 的比特。先利用 $lsb = z \& (\sim z + 1)$ 取出 z 最右侧的 1，该位对应原整数的第一个 0 比特。再使用异或 $z = z \oplus lsb$ 清除 z 中这个最右侧的 1，舍弃第一个 0 对应的比特位。再把处理过后的 z 代入提取最右置 1 比特的表达式 $z \& (\sim z + 1)$ ，得到的结果就是原整数第二个 0 比特对应的掩码；若没有第二个 0，运算直接返回 0。

P8 思路

题目要求是获得全部 32bit 中 1 的个数的奇偶性。因为异或运算不改变 1 的个数奇偶的性，所以通过多次错位的异或，可以得到 1 个数的奇偶性

将 32 位与右移 16 位的值异或，把奇偶信息压缩到低 16 位；再依次右移 8、4、2、1 位做异或，不断减半，把全部信息压缩至最低的 1bit。此时前 31 位的信息无效，所以 $x \& 1$ 取出最低比特，此时，该比特为 1 代表原始数有奇数个 1，0 代表偶数个 1。

使用逻辑非!对结果取反，实现个 1 输出 1，奇数个 1 输出 0，返回最终值。

P9 思路

使用 $\text{rot} = n \& 31$ ，将移位范围约束在 0-31，应对输入 n 数值很大的情况。

将原始数字左移 $32 - \text{rot}$ 位，把从右侧被挤出的 rot 个比特移动至高比特区域。

因为要消除有符号数算术右移时高位自动填充的符号位 1，把算术右移转换为逻辑右移，构造一个掩码 mask ；将左移得到的片段与逻辑右移后的高位片段进行按位或运算，拼接两部分比特，得到循环右移结果。

一开始直接使用算术右移 $x \gg \text{rot}$ 。由于输入为负数时，C 语言有符号数会执行算术右移，高位全部补符号位 1，测试负数大移位用例时输出结果错误。

所以在 ai 的辅助下尝试构造掩码，用来清除算术右移补进来多余的 1 比特。但该掩码在 $n=0$ 时程序输出错误，无法保留原始数值。所以我又多次修改掩码计算公式，使用 $\sim((1 \ll 31) \gg \text{rot} \ll 1)$ 生成掩码，通过所有测试用例。

P10 思路

题目需要将非负整数 x 舍入到最近的 2^n 倍数，当数值恰好处于两个倍数正中间时，选取商为偶数的结果。首先计算步长 $\text{pow2} = 2^n$ ，构造掩码把整数 x 的低 n 位全部清零，得到向下对齐的基准值 base ；再计算余数 rem ，代表原数超出基准 base 的数值大小。

接下来判断是否需要向上进位：将余数 rem 与步长的一半 half 做对比。首先计算余数与 half 的差值，提取差值的符号位并判断差值不为零，以此判断余数是否严格大于 half ，若满足该条件则需要向上进位。

再处理中点边界情况：当余数恰好等于 half 时，读取基准 base 除以 2^n 之后结果的最低比特位；若该比特为 1，则向上进位，若为 0 则不进位，以此实现中点处向偶数商舍入的规则。不能使用 if-else 分支语句，因此将上述两组判断条件转化为掩码：条件成立生成全 1 掩码 -1，条件不成立生成 0 掩码。将掩码和步长 pow2 做按位与运算，得到是否需要叠加的增量。最后将基准 base 与增量做加法运算，得到舍入完成后的最终结果。

P11 思路

因为直接计算 $(x+y)/2$ ， x 与 y 相加会发生整数溢出，因此使用 Knuth 算法。 $x \& y$ 可以取出 x 、 y 同为 1 的比特， $x \wedge y$ 可以取出两者不同的比特，异或结果算术右移 1 位后，与 $x \& y$ 相加，得到向负无穷舍入的基础中点。

通过异或结果最低位可以判断是否出现 0.5 小数。要实现向 x 舍入，需要判断 $x > y$ 。最初尝试直接做减法 $x - y$ 比较大小，但输入 INT_MIN 边界值时减法发生溢出，测试出错。

为消除减法溢出，将数值强制转换为 `unsigned` 无符号类型，使用比较运算符完成大小判断。测试发现该方案使用题目禁止的比较操作，边界用例仍然报错，方案不可行。

最后使用位运算完成大小判断。提取 x 、 y 符号位，符号不同时正数更大；符号相同时，通过取反加 1 构造 $-y$ ，相加得到差值，提取差值符号位来判断大小。

得到 x 大于 y 的标记。当出现 0.5 小数且 x 更大时，基础中点加 1，否则保持原值，完成向第一个参数 x 舍入，返回结果。

P12 思路

因为本题不能直接用 $<$ 、 $>$ 判断大小，所以用位运算实现整数大小比较。两个整数做减法 $y + \sim x + 1$ ，等价于 $y - x$ 。

若结果为负数，符号位为 1，代表 $y < x$ ；若结果非负，符号位为 0，代表 $y \geq x$ 。

但减法在两个数符号不同时会发生整数溢出，结果失效。因此需要分两种情况处理：

两数符号不相同：直接由符号位就可以判断大小，不需要做减法。

两数符号相同：此时减法不会发生溢出，可以安全执行减法，再取符号位判断大小。

函数需要判断两种成立情况：

当区间为 $[a,b]$ ($a \leq b$)：x 需要大于等于 a，同时小于等于 b；

当区间为 $[b,a]$ ($b \leq a$)：x 需要大于等于 b，同时小于等于 a。

两种条件任意一种成立，就代表 x 落在区间内。

第一轮调试：溢出问题最开始直接使用减法做大小判断，没有区分符号。当一正一负时减法发生溢出，测试样例输出结果完全错误。

修改：增加符号位判断，区分同号、异号分支。异号直接用符号位判断大小，同号才执行减法，规避溢出。

第二轮调试：返回值不为 0/1 位运算直接运算结果会得到全 1 或者全 0，不是标准的逻辑值 0 和 1，部分测试点报错。

修改：增加双重取反!!，把运算结果归一为 0 或 1，保证返回结果只有 0、1。

第三轮调试：min/max 边界选择逻辑出错，尝试通过掩码选择上下边界，掩码构造错误，导致上下界颠倒，部分测试用例得到相反结果。

修改：放弃掩码选择 min、max 的写法，改为分别写出两套区间判断逻辑，对 $[a,b]$ 、 $[b,a]$ 两种区间分别做判断，再做或运算。

P13 思路

最初直接使用 $(x << 2) + x$ 计算 $5 * x$ ，计划先完成运算，再通过符号位判断是否出现溢出。但 C 语言中，有符号整数左移溢出属于未定义行为，运算结果不可预测。即便后续流程不一定使用该运算结果，只要表达式存在于代码中，编译器就会执行计算，干扰溢出判断逻辑，导致部分边界测试用例运行错误。

之后尝试阈值预判的方案，预先设定临界阈值，在执行乘法运算前，判断数值扩大五倍后是否会超出 int 的表示范围。但阈值比较过程中使用的加法运算仍会发生有符号数溢出，部分边界场景判断失效，依旧无法通过全部测试。

参考仓库 <https://github.com/Chi-Shan0707/DataLab> 的实现思路：分为两步完成溢出检测。首先计算 $x4 = x << 2$ ，将 x4 右移 2 位后与原始 x 做异或对比，若二者不相等，说明 $x << 2$ 移位过程发生溢出，将溢出标记 over 置 1。接着计算 $res = x4 + x$ 得到五倍运算结果，再对加法阶段做溢出检查。移位溢出、加法溢出任意一种情况发生，就将 over 标记置 1。利用 $\sim over + 1$ ，将 0 或 1 转换为全 0 或全 1 的掩码。构造饱和值 INF，依据输入 x 的正负，溢出时正数返回 int 最大值，负数返回 int 最小值。

P14 思路

一开始我尝试直接计算 $x + y + z$ ，但是有符号整型加法溢出属于 C 语言未定义行为，无法直接获取数学意义上真实求和结果，因此不能直接做大小比较。

最初方案：把三数加法拆分为两次两数加法，将每一步得到的溢出标记直接相加，依靠相加后的结果判断整体溢出。但是该方法存在漏洞，第一次加法溢出后的中间结果本身是未定义值，把它带入第二步溢出判断再做算术相加，遇到边界输入（全部输入为 INT_MIN）时输出错误。

参考 <https://github.com/Chi-Shan0707/DataLab> 的实现方案：依旧将求和拆分为两步加法， $sum1 = x + y$ ， $sum2 = sum1 + z$ 。依据两数加法溢出条件，两个正数相加结果为负代表正溢出；两个负数相加结果为正代表负溢出。分别检测两次加法的正溢出标记、负溢出标记。只要两次加法中任意一次发生正溢出，整体判定正溢出；只要任意一次发生负溢出，整体判定负溢出。利用位运算生成返回值。发生正溢出返回 1，发生负溢出返回 -1，没有溢出返回 0。

P15

把输入拆成符号、阶码、尾数三部分。首先判断阶码，如果阶码为 0xff，代表 NaN 或无穷，直接返回原数。

当阶码等于 0，属于非规格化数。将尾数乘 3，取出低两位判断舍入条件，得到舍入位。将乘积右移 1 位，再加上舍入位。如果结果超出 23 位尾数范围，则要补上隐藏的 1，最后拼接符号输出。

处理规格化数，将尾数或上隐藏 1 后乘以 3。判断乘积大小，分两种情况做舍入：若乘积较大，阶码自增 1，按规则计算舍入位，将乘积右移 2 位并加上舍入位；否则计算舍入位，乘积右移 1 位加上舍入位。处理后如果结果超出 23 位，继续右移同时阶码加 1。

判断阶码，如果阶码超过可表示范围，返回同符号无穷。否则拼接符号、阶码、尾数得到结果。

P16

将输入拆分为符号、阶码、尾数。首先判断阶码，若是 NaN 或者无穷，直接返回输入。计算真实指数 e 。当 $e < -1$ ，绝对值小于 0.5，直接返回符号位实现舍入为 0，保留负零。 $e = -1$ 对应数值在 0.5 附近：尾数非零就进位到 1.0；正好 0.5 时向偶数舍入到 0，只返回符号。指数大于等于 23 时，小数部分为 0，已经是整数，直接返回原数。

其余情况，算出需要舍弃的小数位数 shift，构造 mask 取出小数部分 low。half 代表一半阈值。舍入条件：小数部分大于一半直接进位；等于一半时，看整数最低位，奇数才进位，实现向偶数舍入。把输入小数部分全部清零，再把进位位左移对应位数加上，完成舍入。

调试返工：一开始没有单独处理 $e = -1$ 的特殊分支，0.5 附近对半值舍入结果出错。一开始直接对尾数做加法，没有利用掩码清零尾部，会残留小数比特。忽略 e 小于 -1 的情形，无法正确输出 -0，会把负零变成正零。区分大于一半、等于一半两种舍入条件，实现向偶数舍入。

P17

单精度浮点数由 1 位符号、8 位阶码、23 位尾数构成。尾数只能保存 23 位有效比特。因此如果整数有效二进制位数 ≤ 23 ，可以直接放入尾数；超过 23 位就必须舍弃低位，同时做向偶数舍入。

如果输入是 0，直接返回 0。对于 INT_MIN，因为取负会溢出，无法转为正数，直接返回它对应的浮点编码。提取符号位，把负数转为正数。想要构造阶码，反向寻找整数最高位 1 的位置，该位置就是原始指数 exp。

若 $\text{exp} \leq 23$ ，有效位全部能放进 23 位尾数，只需移位对齐尾数，拼接符号、偏移后的阶码直接输出。如果 $\text{exp} > 23$ ，有效比特超出 23 位，尾数存不下全部数据，必须丢掉低位。

根据向偶数舍入规则：把要丢掉的低位取出，低位大于一半就进位；刚好一半时看整数最低位，奇数进位偶数不进位。整数右移舍弃低位，加上舍入进位。反向考虑浮点数尾数不能超过 23 位：如果进位让有效位跑到第 24 位，尾数存不下，就要整体右移 1 位，同时指数加 1。最后把符号、阶码、尾数拼接，得到浮点数结果。

P18

把 32 位拆成多组，先在小分组内保存每组 1 的计数，再不断合并相邻分组的计数值，最后合并得到全部 32 位结果。

首先使用掩码，两两一组，把每 2 位里面 1 的数量，用 2 位保存；接着合并为 4 位一组，把相邻两组 2 位的计数相加，用 4 位存储；继续扩大分组，依次完成 8 位、16 位，最后合并左右两半 16 位的计数值，得到全部 32 位中 1 的总数量。

P19 思路

该函数实现 32 位整数比特反转，将数字最高位与最低位互换。题目不允许书写大于 0xff 的字面常量，各个掩码全部依靠移位与加法运算构造。int 类型负数右移是算术右移，会向高位填充符号位 1，如果不处理会污染结果。不能使用 unsigned 类型和强制类型转换，因此每次算术右移完成后，立刻使用对应掩码按位与，把高位额外生成的 1 全部清除，模拟逻辑右移行为。采用分治交换思想：依次交换 16 位块、8 位块、4 位块、2 位块、相邻 1 比特。每一步用掩码取出目标分组，分别做右移、左移，再用按位或把两部分拼接。